AI-BASED MULTI-MODAL WEB THREAT DETECTION AND SECURITY INTELLIGENCE SYSTEM

Tri-Modal HTML, Content, And Behavioral Intent Analysis for Multi-Class Web Threat Detection

Project ID: J26-SE-316

Project Proposal Report

Individual Component 02
Dilshan.N – IT23250574

B.Sc. (Hons) Degree in Information Technology

Specialized in **Software Engineering**

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

Supervisor: Mr. Eishan Weerasinghe
Co-Supervisor: Ms. Madusha Weerasooriya

September 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Dilshan. N	IT23250574	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Eishan Weerasinghe	2026/09/13	

3

ABSTRACT

Sri Lanka faces a severe surge in multi-class web scams, including phishing, fake e-commerce, and crypto frauds. Sri Lanka CERT reported over 12,650 cyber incidents in 2025, with Kaspersky detecting 9,218 financial phishing attempts in 2024. However, existing detection tools predominantly rely on URL blacklists or heavy OS-level sandboxes, failing to capture localized social-engineering lures and client-side behavioral intents (e.g., form hijacking, clipboard theft) in low-resource languages like Sinhala and Tamil. Furthermore, current HTML analyzers treat static structure and dynamic behavior as isolated pipelines, ignoring multilingual contexts and lightweight client-side anomalies.

This research proposes a novel tri-modal "HTML, Content & Behavior Analysis" component. It fuses cross-lingual NLP (XLM-RoBERTa), structural DOM parsing, and static code-level behavioral intent mapping to detect multi-class threats in English, Sinhala, and Tamil without heavy sandboxing. The methodology employs iterative prototyping—starting with DOM extraction, integrating multilingual transformer models, and performing static JavaScript dataflow analysis—developed via an agile approach. Evaluation involves training ensemble ML models (XGBoost) for multi-class classification and conducting ablation studies to prove the fused tri-modal approach achieves a target ≥95% F1-score, outperforming single-modality baselines by ≥10%.

Within the overarching AI-Based Web Threat Discovery System, this component acts as the core semantic and behavioral engine. It analyzes actual webpage code and text to provide explainable, multi-class threat intelligence, perfectly complementing the system's URL pre-filtering, visual spoofing detection, and heavy sandbox execution modules.

Keywords: Multi-Class Scam Detection, Cross-Lingual NLP, DOM Analysis, Behavioral Intent, Web Threat Intelligence.

ACKNOWLEDGEMENT

The author wishes to express sincere gratitude to the supervisor, Mr. Eishan Weerasinghe, and co-supervisor, [Co-Supervisor Name], for their continuous guidance, constructive feedback, and invaluable insights throughout the development of this research proposal and the ongoing project. Appreciation is also extended to the Department of Software Engineering and the Information Assurance & Security (IAS) Research Group at the Sri Lanka Institute of Information Technology (SLIIT), for the academic resources, laboratory facilities, and technical support provided throughout the research process.

The author would also like to acknowledge the fellow team members - W.A.D.A.K. Appuhamy, M.T.C. Peiris, and M.S.N. Peiris - for their seamless collaboration, knowledge sharing, and contribution towards the shared vision of the Ai-Based Multi-Modal Web Threat Detection And Security Intelligence System

Finally, sincere gratitude is extended to family and friends for their endless encouragement, understanding, and continuous support throughout this research journey.

TABLE OF CONTENT

Table of Contents

DECLARATION 2

ABSTRACT 3

ACKNOWLEDGEMENT 4

TABLE OF CONTENT 5

LIST OF TABLES 8

LIST OF FIGURES 9

LIST OF ABBREVIATIONS 10

LIST OF APPENDICES 11

1. INTRODUCTION 12

1.1. Background and Context 12

1.2. Literature Review 13

1.2.1. HTML/DOM Structural Analysis for Web Threat Detection 13

1.2.2. Multilingual NLP for Social Engineering and Scam-Lure Detection 14

1.2.3. JavaScript Behavioral Analysis and Formjacking Detection 14

1.2.4. Deceptive UI and Clickjacking Detection 15

1.2.5. Multi-Class Scam Detection (E-commerce, Crypto, Social Engineering) 15

1.2.6. Existing Systems and Products 16

1.2.7. Comparative Summary of Literature 17

1.3. Research Gap 18

2. OBJECTIVE 19

2.1. Main Objective 19

2.2. Specific Objectives 19

3. METHODOLOGY 20

3.1. Research Methodology / Research Design 20

3.2. Data / Participants / Experimental Inputs 21

The research will use three main categories of experimental inputs to train, evaluate, and validate the HTML, Content & Behavior Analysis component. As stated in the project's ethical clearance, this component does not involve human participants, surveys, or interviews; all inputs are derived strictly from web artifacts, text corpora, and synthesized feature datasets. . 21

3.2.1. Webpage Artifacts and Behavioral Traces 21

3.2.2. Multilingual Scam-Lure Text Corpus 21

3.2.3. Labeled Multi-Class Scam Dataset 22

3.3. Baseline / Benchmark 22
3.4. Evaluation Metrics 23
3.5. Validation Strategy 24
3.6. Development Approach 25
4. HIGH-LEVEL SYSTEM ARCHITECTURE 26
4.1. Overview of Proposed Solution and Its Components 26
4.2. System Diagram of Components 27
4.3. Individual Component and Its Parts 28
4.3.1. Headless Artifact Fetcher 28
4.3.2. DOM Structural Parser 28
4.3.3. Multilingual NLP Engine 29
4.3.4. Static Behavioral Intent Analyzer 29
4.3.5. Multi-Modal Fusion and Classification Engine 29
4.3.6. Explainable Threat Intelligence Generator 29
4.4. Overall Integration 30
4.5. Individual Responsibility Boundary 32
5. USER REQUIREMENTS 33
5.1. Stakeholders / User Personas 33
5.1.1. Institutional Security Analyst / CERT Analyst 33
5.1.2. System Integrator / Platform Operator 33
5.1.3. Everyday Digital Consumer (Indirect) 34
5.2. Functional Requirements 34
5.3. Non-Functional Requirements 35
5.4. Responsible AI Requirements 36
5.5. Trust and Explainability Requirements 36
6. COMMERCIALIZATION PLAN 38
6.1. Target market and customer persona 38
6.2. Problem and Customer Need 38
6.3. Value Proposition 39
6.4. Revenue Model 40
6.5. Cost Estimation 41
6.6. Pricing Strategy 41
6.7. Competitive Advantage 41
7. BUDGET AND JUSTIFICATION 44
7.1. Budget Justification 45

8. WORK BREAKDOWN STRUCTURE (WBS) 46
9. GANTT CHART 47
10. REFERENCES LIST 48
11. APPENDICES 50
Appendix 1 - Supporting Proposal Evidence 50
A. Literature Comparison 50
B. Preliminary Data Requirements 51
Appendix 2 - Mandatory Proposal Evidence and Decision Log 51
Appendix 3 - AI Use Disclosure 53

LIST OF TABLES

Table 1.1: Comparative Summary of Literature 18
Table 3.1: Evaluation Metrics 23
Table 4.1: Individual Responsibility Boundary 32
Table 5.1: Functional Requirements 34
Table 5.2: Non-Functional Requirements 35
Table 6.1: Cost Estimation 41
Table 6.2: Pricing Strategy 41
Table 7.1: Budget form 44
Table 11.1: Literature Comparison for Component 02 51
Table 11.2: Preliminary Data Requirements for Component 02 51
Table 11.3: Mandatory Proposal Evidence and Decision Log 53
Table 11.4: AI Use Disclosure 53

LIST OF FIGURES

Figure 1 System Diagram 27
Figure 2: Overall High-Level System Diagram 30
Figure 3: Work Breakdown structure 46
Figure 4: Gantt chart 47

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
BERT	Bidirectional Encoder Representations from Transformers
CERT	Computer Emergency Readiness Team
DGA	Domain Generation Algorithm
DOM	Document Object Model
F1-Score	F1 Score (Harmonic Mean of Precision and Recall)
FPU	False Positive Rate
GPU	Graphics Processing Unit
HAR	HTTP Archive
IAS	Information Assurance & Security
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
PDPA	Personal Data Protection Act
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
SDG	Sustainable Development Goal
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
WBS	Work Breakdown Structure
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting
XLM-RoBERTa	Cross-Lingual Language Model - RoBERTa

LIST OF APPENDICES

Appendices 1.Supporting Proposal Evidence.....50
Appendices 2. Mandatory Proposal Evidence and Decision Log 51
Appendices 3. AI Use Disclosure 53

1. INTRODUCTION

1.1. Background and Context

Sri Lanka is experiencing a severe escalation in multi-class web scams, including phishing, fraudulent e-commerce, and cryptocurrency frauds, driven by the country's rapid digital transformation. Everyday digital consumers with basic IT skills and institutional security analysts are the primary affected parties. These scams deliberately target vulnerable groups, causing direct financial loss, identity theft, and a profound erosion of public trust in digital banking and e-commerce. The scale of this threat is stark: Sri Lanka CERT reported over 12,650 cybersecurity incidents in 2025, predominantly linked to social media-driven financial scams, while Kaspersky detected 9,218 financial phishing incidents and over 8.6 million web-based cyberthreats in Sri Lanka in 2024 alone.

Current defensive solutions and existing research exhibit critical limitations. Commercial tools rely heavily on reactive URL blacklists, which fail against zero-day threats that survive for only hours. Academic approaches remain fragmented; for example, existing e-commerce fraud detection relies on static HTML features but misses post-load scripted behaviors like fake checkout redirects, while crypto-scam detection often analyzes only advertisement text, ignoring the malicious landing pages themselves. Crucially, these systems lack multilingual support for low-resource languages like Sinhala and Tamil, and they treat static content and dynamic behavior as isolated pipelines, leaving them highly vulnerable to modern evasion techniques such as obfuscated JavaScript, hidden iframes, and form hijacking.

To bridge this gap, the proposed HTML, Content & Behavior Analysis component introduces a novel tri-modal detection engine. It fuses structural DOM parsing, multilingual NLP (utilizing XLM-RoBERTa), and lightweight static JavaScript behavioral intent analysis. This enables the system to proactively detect credential harvesting, sensitive data requests, and deceptive UI patterns across English, Sinhala, and Tamil, identifying malicious actions without relying on outdated blacklists or heavy OS-level sandboxing.

This research is firmly anchored in the Information Assurance & Security (IAS) research cluster, leveraging advanced AI and behavioral feature engineering. Furthermore, it directly advances Sustainable Development Goal 8 (Decent Work and Economic Growth) by protecting household savings and SME revenue, SDG 9 (Industry, Innovation, and Infrastructure) by strengthening resilient digital frameworks, and SDG 16 (Peace, Justice, and Strong Institutions) by empowering national CERT teams and law enforcement with explainable, automated threat intelligence to combat organized cybercrime.

1.2. Literature Review

Research into web-based threat detection has expanded significantly across structural HTML analysis, natural language processing for social engineering, client-side JavaScript behavior, and multi-class scam classification. However, systems that integrate all three modalities—DOM structure, multilingual content, and behavioral intent—within a single lightweight pipeline remain extremely limited. This survey reviews the state of the art across DOM-based phishing detection, multilingual NLP for scam-lure identification, JavaScript behavioral and formjacking analysis, deceptive UI detection, multi-class scam classification, and existing commercial systems, highlighting their contributions, shortcomings, and relevance to the proposed HTML, Content & Behavior Analysis component.

1.2.1. HTML/DOM Structural Analysis for Web Threat Detection

Structural analysis of HTML and the Document Object Model (DOM) has been one of the earliest and most widely studied approaches for identifying phishing and fraudulent webpages, yet most systems focus exclusively on binary classification and ignore deeper behavioral signals.

- Comprehensive Feature-Based Detection. Khonji et al. [1] conducted a foundational literature survey categorizing phishing detection into list-based, heuristic, and machine learning-based methods. They identified that HTML-based features—such as form count, iframe presence, and DOM depth—consistently outperform URL-only lexical features in accuracy. However, as a survey, the work did not propose a new unified model or evaluate multimodal fusion across DOM, text, and behavior. Its taxonomy directly informs the feature engineering stage of the proposed component by establishing which DOM attributes (hidden inputs, external form actions) are most discriminative.
- ML-Based HTML Feature Classification. Rao and Pais [2] developed an efficient feature-based machine learning framework using 30 hand-crafted HTML and URL features, evaluated on the UCI Phishing Websites Dataset. Their Random Forest classifier achieved 99.5% accuracy. The strength lies in its lightweight feature set suitable for real-time deployment. However, the dataset is limited to binary (phishing/legitimate) labels, the HTML features are shallow (e.g., tag counts rather than structural semantics), and the system does not analyze JavaScript behavior or multilingual content. This finding motivates the proposed component's shift from shallow tag counting to deep DOM tree parsing and multi-class classification.
- Fine-Grained Deep DOM Analysis. Zou et al. [3] proposed a fine-grained deep learning approach that converts webpage HTML into visual and structural feature matrices, achieving strong results on phishing detection. Their method captures layout semantics beyond simple tag counting. However, it does not extract semantic text meaning through NLP, cannot detect post-load JavaScript-driven behaviors such as form hijacking, and was evaluated exclusively on English-language phishing samples. This gap directly supports the proposed component's fusion of DOM structural vectors with XLM-RoBERTa text embeddings and static JavaScript dataflow analysis.

1.2.2. Multilingual NLP for Social Engineering and Scam-Lure Detection

Social engineering text—urgency language, fake warnings, and manipulative lures—is the primary attack vector across phishing, crypto scams, and romance frauds, yet NLP-based detection remains overwhelmingly English-centric.

- Cross-Lingual Transformer Representations. Conneau et al. [4] introduced XLM-RoBERTa, a transformer model pre-trained on CommonCrawl data across 100 languages, including Sinhala and Tamil. Evaluated on cross-lingual benchmarks (XNLI, MLQA), it demonstrated superior zero-shot transfer performance to low-resource languages compared to multilingual BERT. The strength is its massive multilingual coverage and robust cross-lingual alignment. However, it was designed for general NLP tasks (text classification, question answering) and has not been fine-tuned for cybersecurity-specific tasks such as scam-lure or urgency-language detection. This directly justifies the proposed component's decision to fine-tune XLM-RoBERTa on a custom-annotated Sinhala, Tamil, and English scam-lure corpus for social-engineering text classification.
- BERT-Based Scam Advertisement Detection. Atondo Siu [5] applied BERT-based text classification to detect cryptocurrency and investment scam advertisements across Bitcointalk, Reddit, YouTube, and app-market listings at scale. The study demonstrated that scam-luring language follows identifiable, evolving patterns tied to market cycles. The method achieved strong precision on advertisement text. However, the system analyzes only the text of advertisements and forum posts, not the actual landing page HTML or JavaScript that victims are redirected to. It is also limited to English. This limitation highlights the need for the proposed component, which extends NLP analysis from advertisement text to the actual webpage content and supports Sinhala and Tamil.
- Romance Scam Text Analysis. de Jong [6] trained classifiers to recognize images reused across dating-scam profiles and subsequently applied NLP to analyze manipulative, trust-building language in scam conversations. The work validated that social-engineering text follows identifiable linguistic patterns. However, the system operates solely on conversation text and profile images in isolation, without analyzing any associated fraudulent payment or investment webpages. This gap motivates the proposed component's approach of detecting social-engineering lures directly on the webpage DOM content rather than in external conversation channels.

1.2.3. JavaScript Behavioral Analysis and Formjacking Detection

Modern scam pages increasingly rely on client-side JavaScript to execute malicious behaviors such as form hijacking, clipboard manipulation, and forced redirects, yet most content-based detectors ignore these signals entirely.

- Large-Scale DOM-Based Behavioral Detection. Lekies et al. [7] analyzed over 25 million real-world web flows to detect DOM-based cross-site scripting (XSS) and malicious client-side behaviors by tracking data flows between DOM sources (user inputs, URL fragments) and sinks (document.write, eval). Their method, evaluated on the Alexa Top 10,000 websites, demonstrated that static dataflow tracking can identify malicious JavaScript behaviors without full dynamic execution. The key strength is its scalability and ability to detect behavioral intent from source code. However, the study focused exclusively on XSS vulnerabilities rather than scam-specific behaviors (form hijacking to external domains, clipboard theft) and did not integrate NLP or DOM structural features. This finding directly

influences the proposed component's adoption of static dataflow tracking to monitor form submission endpoints and detect formjacking without requiring a heavy OS-level sandbox

- Obfuscation and Malicious Script Analysis. Ugarte-Pedrero et al. [8] proposed a machine learning-based system for detecting malicious JavaScript through static analysis of Abstract Syntax Trees (AST) and entropy-based obfuscation scoring. Evaluated on the Hynek Petrak malicious JavaScript corpus and benign scripts from popular websites, the system achieved high detection rates for obfuscated malware loaders and drive-by download scripts. The strength lies in its ability to identify malicious intent even when code is heavily obfuscated. However, it treats all malicious JavaScript as a single binary class and does not differentiate between scam-specific behaviors (e.g., credential harvesting via form hijacking vs. crypto-mining). This limitation supports the proposed component's extension of AST analysis to map specific behavioral intents to multi-class scam typologies.

1.2.4. Deceptive UI and Clickjacking Detection

Attackers frequently employ invisible overlays, hidden iframes, and UI redressing to trick users into unintended interactions, yet few detection systems analyze these deceptive patterns at the DOM level.

- Clickjacking Vulnerability Analysis. Rydstedt et al. [9] conducted the first comprehensive study of clickjacking vulnerabilities across popular websites, analyzing frame-busting scripts and invisible overlay techniques. Evaluated on the Alexa Top 500, they found that over 60% of websites were vulnerable to some form of UI redressing. The study established foundational heuristics for detecting invisible iframes and overlapping elements. However, it focused on vulnerability assessment of legitimate sites rather than detecting actively malicious scam pages, and did not propose an automated real-time classification model. This work informs the proposed component's deceptive UI heuristics, particularly the detection of right-click blocking, hidden overlay elements, and invisible iframe abuse.

1.2.5. Multi-Class Scam Detection (E-commerce, Crypto, Social Engineering)

While individual scam categories have been studied in isolation, no existing system classifies a suspicious webpage across all scam types simultaneously using a unified feature pipeline.

- **Fake E-Commerce Detection.** Sánchez-Paniagua et al. [10] combined HTML tags, textual content, and website images to classify fake online stores, later contributing the ELFW-2031 labeled corpus to standardize evaluation. Their system achieved strong binary classification of fraudulent vs. legitimate e-commerce sites. The strength is the use of multimodal HTML/text/image features. However, it never executes the page or analyzes JavaScript, so it misses fake checkout flows, post-load payment-gateway redirects, and formjacking behaviors. It also does not support multilingual content analysis. This gap is directly addressed by the proposed component, which augments HTML/text features with static JavaScript behavioral analysis and multilingual NLP.

- **Phishing Kit and Infrastructure Analysis**. Oest et al. [11] analyzed over 1,400 phishing kits captured in the wild, studying their internal HTML structure, embedded JavaScript, and server-side PHP scripts. They found that phishing kits follow highly reusable templates with common structural patterns (pre-filled forms, hardcoded credential exfiltration endpoints,

anti-bot scripts). Evaluated across multiple phishing feeds, the study provided critical insights into kit-based attack infrastructure. However, the analysis was retrospective and manual, not automated for real-time detection, and did not incorporate NLP-based content analysis. This finding influences the proposed component's approach of matching extracted DOM/JavaScript signatures against known malicious behavioral patterns for zero-day detection.

1.2.6. Existing Systems and Products

Several commercial and community-driven systems currently attempt to detect phishing and web scams, but each has significant limitations that the proposed component addresses.

- **Google Safe Browsing.** This system maintains a continuously updated blacklist of known malicious URLs using a combination of automated crawling and user reports [12]. It uses machine learning classifiers on URL and page features and is integrated into Chrome, Firefox, and Safari. While it protects billions of users globally, it is fundamentally reactive—a URL must be discovered, reported, and verified before being blacklisted, leaving a critical window where zero-day phishing pages and scam storefronts operate freely. It also does not perform multilingual content analysis or static JavaScript behavioral intent extraction.
- **PhishTank and OpenPhish.** These are community-driven and automated phishing URL feeds that aggregate user-submitted and machine-detected phishing links [13]. They provide open APIs for security tools. Their strength is rapid community-driven reporting. However, they function solely as URL blacklists without analyzing webpage content, HTML structure, or JavaScript behavior. They cannot detect zero-day scams that have not yet been reported, and they provide no multi-class scam classification or explainable evidence.
- **Netcraft Anti-Phishing.** Netcraft employs a combination of URL analysis, page content scanning, and community toolbar reports to identify phishing sites [14]. It serves enterprise clients including banks and government agencies. While it analyzes some page content, it primarily focuses on brand impersonation through logo and domain matching rather than deep DOM structural analysis, JavaScript behavioral intent, or multilingual NLP-based social-engineering detection.
- **Microsoft Defender SmartScreen.** Integrated into Microsoft Edge and Windows, SmartScreen uses URL reputation, application reputation, and heuristic page analysis to warn users about suspicious sites [15]. It serves hundreds of millions of Windows users. Its limitations include a heavy reliance on Microsoft's proprietary URL reputation database, limited transparency in its detection methodology, no support for low-resource languages like Sinhala and Tamil, and no static JavaScript dataflow analysis for detecting formjacking or clipboard manipulation.

Collectively, these existing systems demonstrate that URL reputation and basic page scanning can catch known threats, but they all fail to provide the deep, lightweight, multilingual, tri-modal (DOM + NLP + Behavior) analysis required to detect zero-day, multi-class web scams that use obfuscated JavaScript, localized social-engineering text, and deceptive UI patterns.

1.2.7. Comparative Summary of Literature

Author(s) & Year	Focus Area	Key Contributions	Limitations / Gaps
Khonji et al., 2013 [1]	Survey – Phishing Detection	Taxonomy of list-based, heuristic, and ML-based phishing detection; identified HTML features as promising	No unified multimodal model; no behavioral or NLP analysis
Rao & Pais, 2019 [2]	ML-based Phishing Detection	30 HTML/URL features with Random Forest; 99.5% accuracy on UCI dataset	Binary classification only; shallow HTML features; no JS behavior or multilingual support
Zou et al., 2020 [3]	Deep Learning DOM Analysis	Fine-grained structural feature matrices for phishing detection	No NLP semantic analysis; no JS behavioral detection; English-only
Conneau et al., 2020 [4]	Cross-Lingual NLP	XLM-RoBERTa pre-trained on 100 languages including Sinhala and Tamil	Not fine-tuned for cybersecurity or scam-lure detection
Atondo Siu, 2025 [5]	Crypto Scam Detection	BERT-based classification of crypto scam advertisements at scale	Analyzes only advertisement text, not landing page HTML/JS; English-only
de Jong, 2019 [6]	Romance Scam Detection	Image reuse and NLP-based manipulative language detection	Isolated from webpage/payment page analysis; no HTML/DOM features
Lekies et al., 2013 [7]	DOM-based XSS Detection	Large-scale static dataflow tracking of DOM sources and sinks	Focused on XSS, not scam-specific behaviors; no NLP or DOM structure fusion
Ugarte-Pedrero et al., 2016 [8]	Malicious JS Detection	AST-based static analysis and entropy-based obfuscation scoring	Binary classification only; no multi-class scam typology mapping
Rydstedt et al., 2010 [9]	Clickjacking Analysis	First comprehensive study of UI redressing and frame-busting vulnerabilities	Vulnerability assessment only; no real-time automated classification model
Sánchez-Paniagua et al., 2021 [10]	Fake E-Commerce Detection	Multimodal HTML/text/image classification; ELFW-2031 dataset	No page execution or JS analysis; misses fake checkout redirects; no multilingual NLP
Oest et al., 2017 [11]	Phishing Kit Analysis	Structural analysis of 1,400+ phishing kits; identified reusable templates	Retrospective and manual; no real-time automation or NLP
Google, 2024 [12]	Existing System – Safe Browsing	ML-based URL/page blacklist; protects billions of users	Reactive; no multilingual NLP; no JS behavioral intent analysis

PhishTank / OpenPhish [13]	Existing System – URL Feeds	Community-driven and automated phishing URL aggregation	URL-only blacklist; no content, DOM, or behavior analysis
Netcraft, 2024 [14]	Existing System – Anti-Phishing	URL analysis, content scanning, brand impersonation detection	No deep DOM/JS analysis; no multilingual NLP; no behavioral intent extraction
Microsoft, 2024 [15]	Existing System – SmartScreen	URL reputation, heuristic page analysis; integrated in Edge/Windows	Proprietary; no Sinhala/Tamil support; no JS dataflow analysis

Table 1.1: Comparative Summary of Literature

1.3.Research Gap

The reviewed literature shows that static HTML feature extraction [1], [2], cross-lingual transformer models (XLM-RoBERTa) [4], and JavaScript static analysis [7], [8] **can individually identify** phishing structures, social-engineering text, and malicious scripts. Furthermore, specialized studies have successfully classified fake e-commerce [10] and crypto-scam advertisements [5] using **isolated modalities. However,** these capabilities are **not sufficiently integrated into a single lightweight pipeline** that extracts **code-level "behavioral intent"** (e.g., form hijacking, clipboard manipulation) **without relying on resource-heavy OS-level sandboxes,** while simultaneously **detecting localized social-engineering lures in low-resource languages like Sinhala and Tamil** and mapping features to **distinct multi-class scam typologies** [1], [3], [10].

Therefore, this research will investigate whether a **tri-modal fusion** of cross-lingual NLP, structural DOM parsing, and static JavaScript behavioral intent mapping can provide more accurate, **zero-day, multi-class web threat detection** in English, Sinhala, and Tamil than conventional **single-modality, English-only, and heavy-sandbox-dependent approaches.**

2. OBJECTIVE

2.1.Main Objective

To design, develop, and evaluate a tri-modal HTML, Content, and Behavioral Intent Analysis engine that fuses structural DOM parsing, cross-lingual NLP (XLM-RoBERTa), and static JavaScript dataflow analysis to detect multi-class web threats (Phishing, Fake E-commerce, Crypto/Investment Scams, Malware-Delivery) in English, Sinhala, and Tamil, achieving a target classification F1-score of ≥95% and demonstrating a ≥10% performance improvement over single-modality baselines.

This is measurable because the system's performance will be quantitatively evaluated using multi-class F1-scores, precision, and recall metrics on a custom multilingual dataset, alongside a rigorous ablation study comparing the fused tri-modal approach against isolated DOM-only, NLP-only, and behavior-only baselines.

2.2.Specific Objectives

1. **To design and implement a lightweight DOM structural parser and static JavaScript** analyzer that extracts code-level behavioral intent features (e.g., form hijacking, clipboard manipulation, forced redirects, and deceptive UI patterns) without relying on heavy OS-level sandbox execution.
2. **To develop a cross-lingual social-engineering** detection module using a fine-tuned XLM-RoBERTa transformer model to accurately identify psychological urgency, fake warnings, and scam lures in English, Sinhala, and Tamil.
3. **To construct and annotate a localized, multilingual scam-lure corpus** (English, Sinhala, Tamil) through data augmentation and manual verification to support the training and evaluation of the NLP module for low-resource languages.
4. **To design a multi-modal feature fusion mechanism** that integrates XLM-RoBERTa text embeddings, DOM structural vectors, and behavioral intent flags into a unified feature representation for downstream multi-class threat classification.
5. **To train and evaluate ensemble machine learning models** (e.g., XGBoost, Random Forest) for multi-class scam categorization, and conduct an ablation study to quantify the individual contribution and performance gain of each modality (HTML, Content, Behavior).
6. **To implement an explainable threat intelligence** output generator that translates the fused model's predictions into structured, human-readable risk findings (e.g., JSON-formatted alerts detailing credential harvesting, obfuscated JS, and multilingual urgency text) for downstream system integration.

3. METHODOLOGY

3.1. Research Methodology / Research Design

This research adopts an **Experimental Design-and-Evaluation Research Methodology** within a **Design Science Research (DSR)** framework to develop and validate a tri-modal framework for multi-class web threat detection. The study centers on architecting an artifact capable of simultaneously parsing client-side DOM/JavaScript behavioral patterns, processing multi-lingual textual scam content (English, Sinhala, and Tamil), and fusing multi-modal feature spaces for precise threat categorization.

The methodology is structured into two iterative phases: **System Artifact Engineering** and **Empirical Quantitative Evaluation**:

- **Nature of the Research Question:** The problem demands addressing both structural code exploitation (client-side execution) and psychological deception across high- and low-resource languages. Because the core challenge lies in the synergistic integration of heterogeneous threat signals without heavy sandboxing overhead, an iterative design-and-evaluation approach enables systematic benchmarking of individual feature pipelines and joint multi-modal fusion.
- **Type of Data:** The study handles high-dimensional, multi-modal, and multilingual datasets. These encompass semi-structured HTML trees, syntactic AST nodes from static JavaScript parsing, and unstructured textual data across English, Sinhala, and Tamil. The experimental setup utilizes an annotated localized corpus augmented to mitigate cross-lingual imbalance and resource scarcity.
- **Experimental Requirements:** Rigorous empirical evaluation requires controlled offline benchmarking against standard baselines (e.g., standalone DOM heuristic parsers, baseline monolingual NLP models, and binary threat classifiers). Evaluation protocols rely on standard machine learning metrics: Precision, Recall, Macro-F1 score, Area Under the ROC Curve (AUC-ROC), and inference runtime/latency to verify lightweight execution capabilities.
- **Relevant Literature:** Established cybersecurity and applied NLP literature demonstrates that unimodal detectors fail against sophisticated adversarial evasion and polymorphic scams. Recent paradigms in multi-modal security research (leveraging pre-trained cross-lingual transformers such as XLM-RoBERTa coupled with structural graph or DOM parsers) emphasize empirical validation across heterogeneous threat corpora to guarantee robustness across real-world evasion tactics.

The selected methodology is appropriate because the research objective is not purely exploratory; it requires the engineering of a functional computational artifact and the subsequent statistical, benchmark-driven validation of its predictive accuracy and performance efficiency.

3.2.Data / Participants / Experimental Inputs

The research will use three main categories of experimental inputs to train, evaluate, and validate the HTML, Content & Behavior Analysis component. As stated in the project's ethical clearance, this component does not involve human participants, surveys, or interviews; all inputs are derived strictly from web artifacts, text corpora, and synthesized feature datasets.

3.2.1.Webpage Artifacts and Behavioral Traces

The primary experimental inputs are the raw webpage artifacts fetched from suspicious and benign URLs. These include:

- HTML source code and parsed DOM structures (forms, inputs, iframes, hidden elements).
- Embedded and external JavaScript code (inline scripts, external .js files).
- Visible webpage text content (filtered from boilerplate code).
- Form submission endpoints, DOM event listeners, and redirect chains.

These artifacts are highly appropriate as they represent the exact attack surface where credential harvesting, formjacking, deceptive UI, and obfuscated malicious behavior occur.

Data Source & Collection: Malicious URLs will be sourced from live threat intelligence feeds (PhishTank, OpenPhish) captured immediately upon reporting, while benign URLs will be drawn from the Tranco Top 1M list. Artifacts will be collected using a headless browser (Playwright) to capture the fully rendered DOM and executed JavaScript context safely. *Preparation Process:* Raw HTML will be parsed and cleaned. JavaScript will be extracted and pre-processed for Abstract Syntax Tree (AST) generation and entropy-based obfuscation scoring. Any sensitive personal data incidentally captured in form fields will be anonymized in accordance with the Personal Data Protection Act.

3.2.2. Multilingual Scam-Lure Text Corpus

To train and fine-tune the cross-lingual NLP module (XLM-RoBERTa), a specialized text corpus is required to detect social-engineering lures, urgency triggers, and fake warnings across multiple languages.

Data Source & Collection: Because public datasets for Sinhala and Tamil phishing text are highly limited, the corpus will be constructed through a semi-supervised approach:

1. Translation and augmentation of existing English phishing/scam text datasets using high-quality LLM translation models.
2. Scraping and aggregating publicly available Sri Lankan social media scam reports, comments, and forums. The corpus will explicitly cover English, Sinhala, and Tamil. *Preparation Process:* The text data will be cleaned, tokenized, and manually annotated by the researcher (and verified by peers/supervisor) into specific categories such as "urgency/threat", "fake reward", "credential request", and "benign". This labeled corpus is critical for enabling the NLP model to understand localized psychological manipulation tactics.

3.2.3. Labeled Multi-Class Scam Dataset

To train and evaluate the final multi-class classification models (XGBoost/Random Forest), a unified dataset fusing DOM features, NLP embeddings, and behavioral flags is required.

Data Source & Sample: The dataset will be synthesized by combining the processed outputs from the webpage artifacts (3.2.1) and the NLP corpus (3.2.2). It will include samples across five distinct classes: Phishing, Fake E-commerce, Crypto/Investment Scam, Malware-Delivery, and Legitimate. Public benchmark datasets, such as ELFW-2031 (for fake e-commerce) and Phish30k (for phishing), will be incorporated to ensure diverse and standardized representation. *Preparation Process:* Feature engineering will be applied to convert DOM structures into structural vectors, text into XLM-RoBERTa embeddings, and JS behaviors into binary/categorical flags. The final dataset will be balanced using techniques like SMOTE if class imbalance is detected, and split into training, validation, and testing sets (e.g., 70/15/15 split) to facilitate rigorous model evaluation and the planned ablation studies.

3.3. Baseline / Benchmark

The proposed component will be compared with two primary baseline approaches to validate the necessity of the tri-modal fusion and multilingual capabilities.

Baseline 1 — Single-Modality HTML/DOM Structural Analysis The first baseline will use traditional machine learning models trained exclusively on static HTML and DOM structural features (e.g., tag counts, form presence, hidden elements, URL length) without any NLP text analysis or JavaScript behavioral tracking. This baseline is important because it represents the conventional static phishing detection approach. Comparing against it will demonstrate whether adding semantic NLP and lightweight behavioral intent extraction provides a significant performance gain over shallow structural heuristics.

Baseline 2 — English-Only NLP Content Analysis The second baseline will use a standard monolingual transformer model (e.g., standard BERT) to analyze only the visible English text of the webpage, ignoring the DOM structure, hidden elements, and JavaScript behavioral logic. This comparison is crucial to validate the necessity of the proposed multilingual capability (Sinhala and Tamil) and the inclusion of structural/behavioral features. It will prove whether analyzing English text in isolation is sufficient, or if cross-lingual and multi-modal fusion is strictly required to catch localized scams and structural obfuscation.

Proposed Approach The proposed approach will combine: **Structural DOM parsing + Cross-lingual NLP (XLM-RoBERTa for English, Sinhala, and Tamil) + Static JavaScript behavioral intent analysis (form hijacking, clipboard manipulation, deceptive UI) + Multi-modal feature fusion.**

This comparison will empirically validate the core novelty: that fusing DOM structure, multilingual text, and lightweight behavioral intent yields superior, zero-day multi-class scam detection compared to isolated, single-modality, or English-only baselines.

3.4.Evaluation Metrics

The proposed component will be evaluated using both machine-learning classification metrics and technical/system-level metrics.

Metric	Purpose / Why Relevant	Target / Comparison
Multi-Class F1-Score (Macro)	Measures the overall classification performance across all threat categories (Phishing, Fake E-commerce, Crypto Scam, Malware-Delivery, Legitimate), accounting for class imbalance.	**Target:** ≥95%. **Comparison:** Must outperform single-modality baselines by ≥10%.
False Positive Rate (FPR)	Measures the rate of legitimate websites incorrectly flagged as malicious. Critical for security tools to avoid disrupting normal user traffic and eroding trust.	**Target:** ≤2%. **Comparison:** Must be lower than or equal to traditional static HTML-only baselines.
Multilingual NLP F1-Score	Measures the accuracy of social-engineering text detection specifically across the three target languages (English, Sinhala, Tamil).	**Target:** ≥90% (EN), ≥80% (SI/TA). **Comparison:** Must significantly outperform English-only BERT baselines on local languages.
Behavioral Intent Detection Rate	Measures the True Positive Rate for identifying specific static JS/DOM behaviors (e.g., form hijacking, clipboard theft, forced redirects, obfuscation).	**Target:** ≥90%. **Comparison:** Validates the lightweight static extraction against known ground-truth malicious scripts.
Ablation Performance Delta	Measures the exact performance gain achieved by fusing all three modalities (DOM + NLP + Behavior) compared to using them in isolation.	**Target:** ≥10% F1-score improvement over the best single-modality baseline.
Processing Latency	Measures the time required to parse the DOM, run NLP inference, and perform static JS analysis per URL.	**Target:** ≤5 seconds per URL. **Comparison:** Must be significantly faster than the heavy OS-level sandbox execution handled by other components.

Table 3.1: Evaluation Metrics

The primary evaluation measures will be the **Multi-Class F1-Score**, **False Positive Rate**, and **Multilingual NLP F1-Score** because these directly address the core research problem of accurately categorizing diverse, localized web scams without disrupting legitimate traffic. Technical metrics such as **Processing Latency**, **Behavioral Intent Detection Rate**, and the **Ablation Performance Delta** will be used to assess the practicality, reliability, and core novelty (tri-modal fusion) of the lightweight static analysis approach. These evaluation categories are consistent with the evaluation plan in your presentation.

The proposed component is expected to demonstrate significantly better multi-class classification accuracy and localized scam-lure detection than the selected single-modality and English-only

baselines, while maintaining acceptable processing latency and a very low false positive rate. Exact numerical targets will be established after pilot testing and finalized before the main evaluation.

3.5. Validation Strategy

Validation will be performed at three levels: functional, technical, and empirical/integration validation.

Functional Validation The individual modules of the HTML, Content & Behavior Analysis component will be tested to verify that:

- the headless browser and DOM parser correctly extract HTML structures, forms, hidden elements, and iframes;
- the static JavaScript analyzer correctly identifies obfuscation, form hijacking, clipboard manipulation, and forced redirects without relying on heavy sandbox execution;
- the XLM-RoBERTa NLP module correctly tokenizes and classifies social-engineering lures across English, Sinhala, and Tamil;
- the multi-modal fusion engine correctly combines DOM structural vectors, text embeddings, and behavioral intent flags;
- the output generator produces the required structured JSON format (including credential_harvesting, risk_score, and findings) for downstream consumption.

If a specific modality fails (e.g., JavaScript extraction fails due to severe anti-bot cloaking), the system will not generate unsupported predictions. Instead, it will flag the missing modality and adjust the confidence score accordingly, requiring appropriate fallback or reliance on the other modalities.

Technical Validation The system's machine learning models and overall pipeline will be evaluated using:

- multi-class F1-score, precision, recall, and False Positive Rate (FPR);
- multilingual NLP F1-score specifically across English, Sinhala, and Tamil;
- behavioral intent detection rate for specific JS/DOM anomalies;
- processing latency per URL.

The generated multi-class verdicts (Phishing, Fake E-commerce, Crypto/Investment Scam, Malware-Delivery, Legitimate) will be compared against ground-truth labels to determine classification accuracy. The generated JSON findings will be manually spot-checked against the raw HTML/JS to ensure the explainable threat intelligence accurately represents the underlying code evidence.

Empirical and Integration Validation A rigorous empirical evaluation will compare the proposed tri-modal approach against the selected baselines (single-modality HTML-only and English-only NLP).

- An ablation study will be conducted using k-fold cross-validation to quantify the exact performance contribution of each modality (DOM, NLP, Behavior).

- The component's JSON output and risk scores will be integrated with the broader system's decision fusion engine to verify that the explainable threat intelligence (e.g., "Password form submits to external domain" + "Sinhala urgency text detected") correctly informs the final system verdict.

The results will be analyzed to determine whether the proposed lightweight, cross-lingual, tri-modal approach provides measurable improvements in zero-day, multi-class scam detection over conventional static and English-only baselines. All data handling will comply with the project's ethical clearance, ensuring no live phishing infrastructure is interfered with and all incidentally captured personal data is anonymized in accordance with the Personal Data Protection Act.

3.6. Development Approach

The component will follow an iterative prototyping development approach because the proposed tri-modal analysis engine involves complex, interconnected machine learning models (DOM parsing, cross-lingual NLP, and static JavaScript analysis) that require continuous testing, feature refinement, and performance tuning.

Development will be divided into incremental stages:

1. **Requirement and literature analysis** Identify specific HTML/DOM structural features, JavaScript behavioral indicators, and multilingual social-engineering text patterns relevant to multi-class web threats (Phishing, Fake E-commerce, Crypto Scams, Malware-Delivery).
2. **Architecture and data pipeline design** Design the headless browser fetching mechanism, DOM parsing logic, static JavaScript analyzer, and the XLM-RoBERTa integration pipeline for processing English, Sinhala, and Tamil text.
3. **Initial prototype development** Develop the basic DOM extraction module and implement initial heuristic rules for identifying suspicious forms, hidden elements, external iframe abuse, and basic deceptive UI patterns.
4. **NLP and behavioral intent integration** Fine-tune the XLM-RoBERTa model on the custom multilingual scam-lure corpus and implement static JavaScript dataflow tracking and AST analysis to detect form hijacking, clipboard manipulation, forced redirects, and obfuscation.
5. **Multi-modal fusion and classification** Fuse the DOM structural vectors, NLP text embeddings, and behavioral flags into a unified feature representation. Train and evaluate ensemble ML models (XGBoost/Random Forest) for multi-class threat classification.
6. **Testing and refinement** Conduct functional testing, perform the ablation study to quantify each modality's contribution, optimize processing latency, and refine the explainable, structured JSON output generation.
7. **System integration and final evaluation** Integrate the component's structured threat intelligence output with the broader system's decision fusion engine and conduct the final empirical evaluation against the selected single-modality and English-only baselines.

An iterative approach is suitable because the accuracy of the NLP model on low-resource languages (Sinhala/Tamil), the effectiveness of the static JavaScript behavioral extraction, and the optimal feature fusion strategy may require repeated refinement based on empirical testing and ablation study results. It also allows the data collection, model training, and system integration to develop together rather than treating model development as a single final stage.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1.Overview of Proposed Solution and Its Components

The proposed research is implemented as the HTML, Content & Behavior Analysis component within a shared four-component AI-Based Web Threat Discovery and Scam Detection System. The complete platform consists of four complementary components: a URL and Domain Intelligence Analyzer, a Visual Analysis and Brand Impersonation Detector, a Secure Sandbox Behavioral Execution Engine, and the proposed HTML, Content & Behavior Analysis component.

The proposed component acts as the core semantic, structural, and lightweight behavioral analysis engine. It does not rely on heavy OS-level sandbox execution or visual pixel analysis. Instead, it receives raw webpage artifacts (HTML, DOM, JavaScript, and visible text) and analyzes the actual code structure, multilingual content, and client-side execution logic to detect multi-class web threats.

The proposed component combines a headless browser artifact fetcher, DOM structural parsing, cross-lingual NLP (XLM-RoBERTa) for English, Sinhala, and Tamil scam-lure detection, and static JavaScript dataflow analysis for behavioral intent extraction. The system is designed to fuse these three modalities into a unified feature representation to accurately classify threats (Phishing, Fake E-commerce, Crypto/Investment Scams, Malware-Delivery) and generate explainable, structured threat intelligence without relying on reactive blacklists.

The four components are integrated through the system's central decision fusion infrastructure. The URL Analyzer performs initial pre-filtering and passes suspicious URLs to the remaining components. The proposed component consumes the raw webpage artifacts, processes them through its tri-modal pipeline, and provides the resulting multi-class verdict, risk score, and structured JSON findings to the ensemble decision engine, which combines them with visual and sandbox outputs for the final system verdict.

4.2.System Diagram of Components

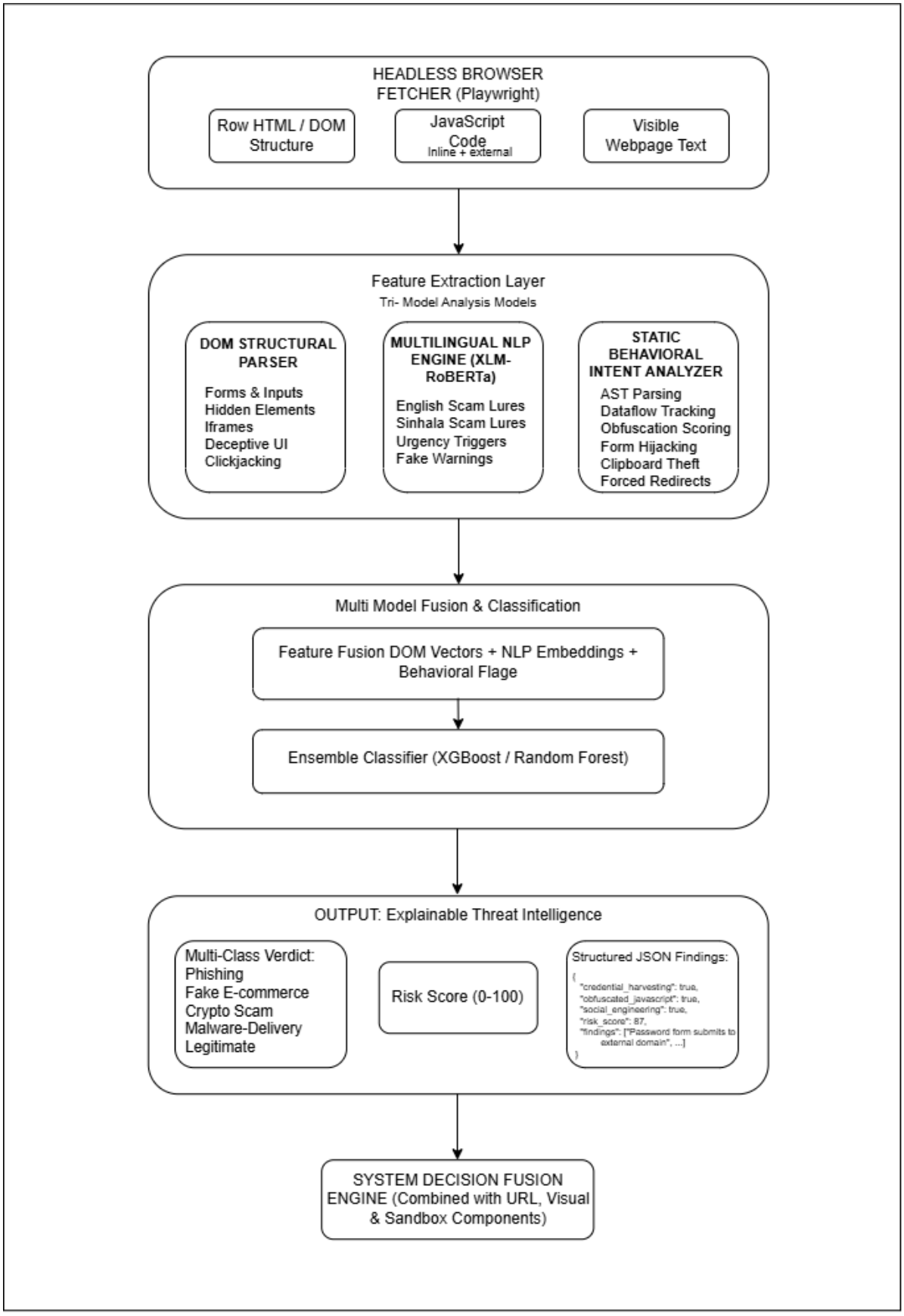


Figure 1 System Diagram

The system diagram illustrates the internal architecture of the HTML, Content & Behavior Analysis component. The component receives three primary inputs from the headless browser fetcher: raw HTML/DOM structure, embedded and external JavaScript code, and visible webpage text content.

These inputs are processed in parallel by three specialized extraction modules. The **DOM Structural Parser** analyzes forms, hidden elements, iframes, and deceptive UI patterns. The **Multilingual NLP Engine** utilizes a fine-tuned XLM-RoBERTa model to analyze visible text and detect social-engineering lures, urgency triggers, and fake warnings across English, Sinhala, and Tamil. The **Static Behavioral Intent Analyzer** performs Abstract Syntax Tree (AST) parsing, entropy-based obfuscation scoring, and static dataflow tracking to detect form hijacking, clipboard manipulation, forced redirects, and malicious download triggers without requiring full sandbox execution.

The extracted features—DOM structural vectors, XLM-RoBERTa text embeddings, and behavioral intent flags—are then passed to the **Multi-Modal Fusion and Classification Engine**. Here, an ensemble machine learning model (e.g., XGBoost/Random Forest) fuses the tri-modal data to generate a multi-class threat verdict (Phishing, Fake E-commerce, Crypto/Investment Scam, Malware-Delivery, or Legitimate) and a calculated risk score.

Finally, the **Explainable Threat Intelligence Generator** translates the model's predictions and feature attributions into a structured, human-readable JSON output (detailing specific findings such as "Password form submits to external domain" or "Sinhala urgency text detected"). This structured JSON, along with the risk score and multi-class verdict, is then passed to the overarching system's Decision Fusion Engine to be combined with visual and sandbox analysis results for the final comprehensive threat assessment.

4.3. Individual Component and Its Parts

Component 02: Tri-Modal HTML, Content, and Behavioral Intent Analysis Engine

Component 02 consists of several interconnected parts designed to extract structural, semantic, and behavioral features from webpage artifacts and fuse them to detect multi-class web threats without relying on heavy OS-level sandboxing.

4.3.1. Headless Artifact Fetcher

The Headless Artifact Fetcher retrieves the raw webpage artifacts required for analysis. Using a headless browser (e.g., Playwright), it captures the fully rendered HTML source code, DOM structure, embedded and external JavaScript, and visible webpage text.

This layer provides the foundational input for the downstream analysis modules, ensuring that dynamically loaded content and external resources are captured before static parsing begins.

4.3.2. DOM Structural Parser

The DOM Structural Parser analyzes the HTML and Document Object Model to extract structural features. It identifies suspicious forms, input fields (such as passwords, OTPs, and card numbers), hidden elements, iframe abuse, and external resource loading.

This module maps structural anomalies to specific threat indicators, such as credential harvesting forms or deceptive UI patterns like clickjacking overlays, invisible inputs, and right-click blocking.

4.3.3. Multilingual NLP Engine

The Multilingual NLP Engine analyzes the visible webpage text to detect social-engineering lures and psychological manipulation. Utilizing a fine-tuned XLM-RoBERTa transformer model, it identifies urgency triggers, fake warnings, and scam-luring language across English, Sinhala, and Tamil.

Instead of relying on simple keyword matching, this layer generates semantic text embeddings and a scam-lure confidence score, enabling the detection of localized, low-resource language scams that bypass traditional English-only filters.

4.3.4. Static Behavioral Intent Analyzer

The Static Behavioral Intent Analyzer examines JavaScript code and DOM event listeners to detect malicious client-side behaviors without requiring full dynamic execution. It performs static dataflow tracking, Abstract Syntax Tree (AST) parsing, and entropy-based obfuscation scoring.

This layer identifies specific behavioral intents such as form hijacking (submitting data to external domains), clipboard manipulation, forced meta/JS redirects, automatic download triggers, and notification abuse.

4.3.5. Multi-Modal Fusion and Classification Engine

The Multi-Modal Fusion and Classification Engine combines the outputs from the DOM, NLP, and Behavioral modules. It fuses DOM structural vectors, XLM-RoBERTa text embeddings, and behavioral intent flags into a unified feature representation.

An ensemble machine learning model (e.g., XGBoost or Random Forest) processes this fused representation to classify the webpage into specific threat categories (Phishing, Fake E-commerce, Crypto/Investment Scam, Malware-Delivery, or Legitimate) and calculate an overall risk score.

4.3.6. Explainable Threat Intelligence Generator

The Explainable Threat Intelligence Generator translates the classification model's predictions into structured, human-readable evidence. It maps the most influential features to specific attacker techniques and generates the final JSON output.

This layer ensures that the downstream decision fusion engine receives not only a multi-class verdict and risk score but also explicit, explainable findings (e.g., "Password form submits data to an external domain" or "Obfuscated JavaScript detected") to support transparent and actionable threat intelligence.

4.4.Overall Integration

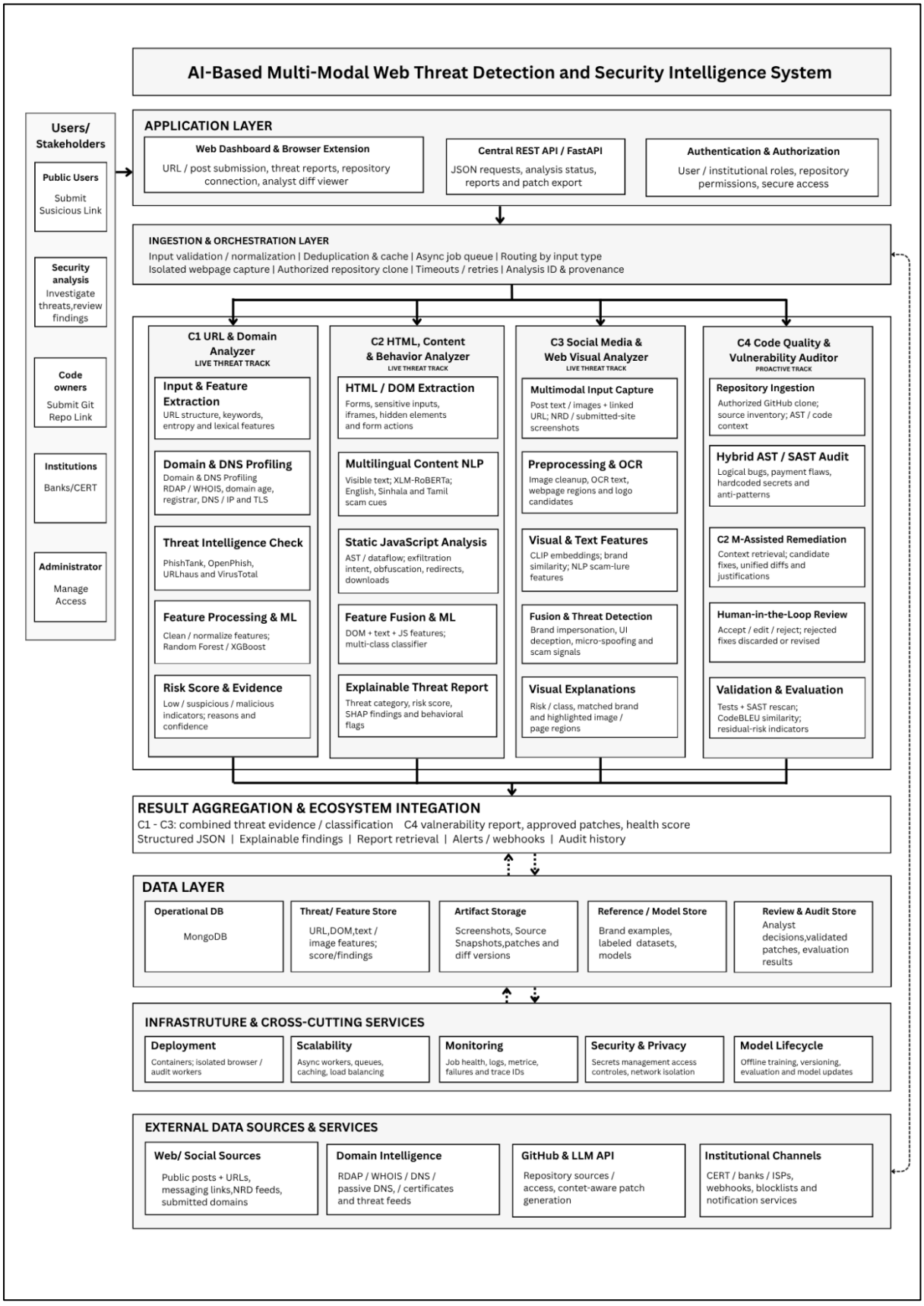


Figure 2: Overall High-Level System Diagram

Component 02 (C2) has both external dependencies and inter-component dependencies within the AI-Based Multi-Modal Web Threat Detection System. Its external dependencies include raw webpage artifacts (HTML source, DOM trees, embedded JavaScript, and visible text) provided by the Ingestion & Orchestration Layer, as well as external threat intelligence feeds and multilingual NLP resources (XLM-RoBERTa) required for model training and inference. Persistent storage via the Data Layer's Threat/Feature Store is also used to maintain extracted DOM/text features, model weights, and historical analysis results.

The primary inter-component dependency is between Component 02 and the **Result Aggregation & Ecosystem Integration** layer. Component 02 operates as an independent, parallel analysis track. After processing the webpage artifacts, C2 exposes its structured JSON output—containing the multi-class threat verdict, risk score, SHAP findings, and behavioral flags—through the internal API. The Result Aggregation layer then consumes this output and combines it with the URL/domain intelligence from C1, the visual and brand-impersonation evidence from C3, and the vulnerability reports from C4 to form a unified, comprehensive threat assessment. These downstream aggregation and final verdict activities remain the responsibility of the central fusion engine rather than being performed by C2.

The planned integration will use REST-based APIs and JSON-formatted communication, supported by a shared Analysis ID and provenance tracking. This ensures that the specific HTML/JS artifacts analyzed by C2 are correctly linked to the URL analyzed by C1 and the screenshots analyzed by C3. The Ingestion & Orchestration Layer utilizes async job queues to route inputs, allowing C2 to process its tri-modal analysis concurrently with the other components. API-based integration keeps the components strictly modular: C2 can update its NLP models or DOM parsing logic without disrupting the visual analysis in C3 or the URL profiling in C1.

The architecture follows a decoupled, parallel-processing runtime approach. When a suspicious URL is submitted via the Application Layer, the Ingestion Layer captures the isolated webpage artifacts and routes them to C2. C2 independently executes its DOM extraction, multilingual NLP, and static JavaScript analysis. Once C2 completes its evaluation, it pushes the resulting JSON payload to the Result Aggregation layer. The normal update flow is therefore: new URL submission -> isolated artifact capture -> parallel C2 analysis -> JSON output to Aggregation -> unified system verdict. This ensures that a delay in C3's heavy visual processing or C4's code audit does not stall C2's lightweight behavioral analysis, maintaining a clear separation between individual modality analysis and the final system-wide decision logic.

4.5. Individual Responsibility Boundary

The following table defines the responsibility boundary of Component 02 and clarifies the activities shared with the other components.

Activity / Deliverable	My Responsibility	Shared Responsibility	Other Member
Headless Artifact Fetcher & DOM Structural Parser	✓		
Multilingual NLP Engine (XLM-RoBERTa)	✓		
Static Behavioral Intent Analyzer (JS/AST/Dataflow)	✓		
Multi-Modal Fusion & Ensemble Classification	✓		
Explainable JSON Threat Intelligence Output	✓		
Multilingual Scam-Lure Corpus Collection & Annotation	✓		
Integration with URL, Visual, and Sandbox components		✓	
URL & Domain Intelligence Analysis			Component 01: W.A.D.A.K. Appuhamy
Code Quality & Vulnerability Auditor with LLM-Assisted Remediation			Component 04: M.S.N. Peiris
Social Media & Web Visual Threat Analyzer			Component 03: M.T.C. Peiris
Final integration of all four components		✓	

Table 4.1: Individual Responsibility Boundary

Responsibility Boundary

The primary responsibility of the researcher is the design, development, and evaluation of Component 02. This includes the headless artifact fetching, DOM structural parsing, multilingual NLP engine, static behavioral intent analysis, multi-modal fusion, and the generation of explainable JSON threat intelligence.

Integration with the other components is a shared responsibility because Component 02 requires the pre-filtered suspicious URLs from Component 01 and must provide its structured JSON findings to the overarching decision fusion engine alongside the visual and sandbox components. The underlying URL/domain lexical intelligence, visual brand verification, and heavy OS-level sandbox execution mechanisms remain outside the individual responsibility of Component 02

5. USER REQUIREMENTS

The user requirements for Component 02 were derived from the shared team requirements elicitation activities, including stakeholder discussions among the project team and supervisor consultation, together with the documented user personas and the identified research gaps. The primary user requirement for Component 02 is that the system must accurately analyze raw webpage artifacts and output structured, explainable, multi-class threat intelligence without relying on heavy sandbox execution or reactive blacklists. The requirements also reflect the need for lightweight processing, cross-lingual scam-lure detection, and seamless integration with the platform's decision fusion engine.

The shared requirements identify that institutional security analysts require accurate, multi-class threat categorization (distinguishing between phishing, fake e-commerce, crypto scams, and malware-delivery) rather than simple binary alerts. Platform-level requirements additionally identify the need for structured JSON outputs, graceful handling of anti-bot evasion, and integration with the URL, visual, and sandbox components.

5.1. Stakeholders / User Personas

5.1.1. Institutional Security Analyst / CERT Analyst

The institutional security analyst is the primary direct consumer of Component 02's outputs. This user requires fast, accurate, and explainable threat intelligence to investigate suspicious URLs circulating on social media. The analyst requires:

- accurate multi-class classification of web threats (not just binary phishing/legitimate);
- structured, human-readable JSON findings that explain *why* a page was flagged (e.g., "Password form submits to external domain");
- detection of localized social-engineering lures in English, Sinhala, and Tamil;
- rapid processing times without the delays associated with heavy OS-level sandboxing;
- sufficient technical evidence to verify the AI's reasoning before initiating takedown procedures.

5.1.2. System Integrator / Platform Operator

The system integrator is a secondary stakeholder responsible for ensuring that Component 02 operates correctly within the wider threat discovery platform. The operator requires:

- reliable API communication between Component 02 and the overarching Decision Fusion Engine;
- graceful degradation and clear error reporting when a webpage employs heavy anti-bot cloaking that blocks JavaScript extraction;
- secure, isolated handling of scraped malicious artifacts to prevent accidental execution or data leakage;

- monitoring of system performance, processing latency, and model drift.

5.1.3. Everyday Digital Consumer (Indirect)

The everyday digital consumer is an indirect stakeholder who benefits from the final system verdict blocking scams before they reach them. The consumer requires:

- that legitimate local e-commerce stores, banking portals, and news sites are not falsely blocked or flagged as malicious;
- protection against zero-day scams that bypass traditional blacklists. The shared requirements identify the need for a very low False Positive Rate (FPR) to maintain public trust in digital services and prevent disruption to legitimate local businesses.

5.2.Functional Requirements

The following functional requirements are scoped specifically to Component 02.

ID	Functional Requirement
FR1	The system shall accept raw webpage artifacts (HTML source, DOM structure, JavaScript code, and visible text) from the headless browser fetcher.
FR2	The system shall parse the DOM to extract structural features, including forms, hidden elements, iframes, and deceptive UI patterns.
FR3	The system shall analyze visible webpage text using a fine-tuned XLM-RoBERTa model to detect social-engineering lures and urgency triggers in English, Sinhala, and Tamil.
FR4	The system shall perform static JavaScript analysis (AST parsing, dataflow tracking, and obfuscation scoring) to detect behavioral intents such as form hijacking, clipboard manipulation, and forced redirects.
FR5	The system shall fuse DOM structural vectors, NLP text embeddings, and behavioral intent flags into a unified feature representation.
FR6	The system shall classify the webpage into specific multi-class threat categories (Phishing, Fake E-commerce, Crypto/Investment Scam, Malware-Delivery, or Legitimate) using an ensemble ML model.
FR7	The system shall generate a structured JSON output containing the multi-class verdict, a calculated risk score, and an array of human-readable explainable findings.
FR8	The system shall gracefully handle anti-bot cloaking by flagging missing modalities (e.g., blocked JS) and adjusting the confidence score rather than crashing or generating false negatives.
FR9	The system shall pass the structured JSON threat intelligence to the integrated Decision Fusion Engine through the agreed platform API interface.
FR10	The system shall anonymize any incidentally captured personally identifiable information (PII) within scraped form fields before processing or logging.

Table 5.1: Functional Requirements

These requirements are consistent with the documented Component 02 requirement to deliver a tri-modal, cross-lingual, and explainable multi-class threat detection engine.

5.3.Non-Functional Requirements

Following requirements define measurable, testable quality characteristics for Component 02.

ID	Requirement	Target / Verification
NFR1 - Response Time	The component shall process a webpage and output the structured JSON within an acceptable latency, significantly faster than heavy sandbox execution.	Measure end-to-end processing latency; target **≤ 5 seconds** per URL during performance testing.
NFR2 - Accuracy	The multi-class classification model shall achieve high predictive performance across all threat categories.	Evaluate using k-fold cross-validation; target **≥ 95% Macro F1-Score** and **≤ 2% False Positive Rate**.
NFR3 - Security	The headless browser and static analysis modules shall operate in an isolated environment to prevent accidental execution of downloaded malware payloads.	Security testing and inspection of the Playwright sandbox and AST parser isolation mechanisms.
NFR4 - Privacy	The system shall minimize the exposure of personally identifiable information incidentally captured in malicious forms.	Privacy review and testing to verify PII anonymization in logs and JSON outputs, complying with the PDPA.
NFR5 - Scalability	The component shall support concurrent URL analysis without unacceptable degradation of processing latency.	Conduct load testing using an agreed concurrent-request level (e.g., 50 URLs/sec).
NFR6 - Maintainability	The DOM parser, NLP engine, and JS analyzer shall be modular so that individual models or extraction logic can be updated independently.	Architecture and implementation inspection.
NFR7 - Localization	The NLP module shall maintain high accuracy for scam-lure detection across all supported target languages.	Evaluate multilingual test sets; target **≥ 90% F1-Score for English** and **≥ 80% F1-Score for Sinhala and Tamil**.

Table 5.2: Non-Functional Requirements

5.4. Responsible AI Requirements

Component 02 shall incorporate responsible-AI requirements to reduce the risks associated with automated threat detection and malicious content handling.

- **RAI1 – Privacy:** The system shall anonymize any incidentally captured PII in scraped form fields and shall avoid exposing unnecessary user information in generated JSON findings or logs.
- **RAI2 – Fairness and Bias:** The system shall monitor classification outputs for potential bias against legitimate local small-to-medium enterprise (SME) e-commerce sites. Where dataset bias is identified, the system shall apply mitigation techniques to prevent the disproportionate flagging of legitimate local businesses.
- **RAI3 – Safety:** The component shall strictly perform static and lightweight dynamic analysis. It shall *not* execute downloaded files, interact with live malware command-and-control (C2) servers, or trigger actual financial transactions during analysis.
- **RAI4 – Transparency:** The system shall clearly indicate in the JSON output which modality (DOM, NLP, or JS Behavior) contributed most heavily to the final risk score, preventing the model from acting as an unexplainable "black box."
- **RAI5 – Auditability:** All processed URLs, extracted features, and generated JSON findings shall be logged securely so that appropriate threat intelligence can be reviewed and audited by security analysts.
- **RAI6 – Human Oversight:** The detailed JSON findings shall provide sufficient technical evidence to allow human security analysts to verify the AI's reasoning and override the verdict (e.g., whitelisting a false positive) before initiating automated takedown actions.

5.5. Trust and Explainability Requirements

Trust and explainability are central to Component 02 because the research specifically investigates whether fusing tri-modal features can produce accurate, zero-day threat intelligence that security analysts can trust and act upon.

- **TE1 – Evidence Grounding:** Every generated risk score and multi-class verdict shall be grounded in the actual extracted DOM, NLP, or JS features. The system shall not generate unsupported alerts.
- **TE2 – Explanation Clarity:** The system shall translate technical feature attributions (e.g., SHAP values or feature importance weights) into understandable, human-readable strings in the JSON "findings" array (e.g., "Obfuscated JavaScript detected" rather than "Feature_42 > 0.8").
- **TE3 – Explanation Faithfulness:** The generated JSON findings shall accurately reflect the actual code and text analyzed. Explanations that fail the faithfulness check (e.g., hallucinating a hidden iframe that does not exist in the DOM) shall be filtered out.

- **TE4 – Evidence Visibility:** The JSON output shall provide specific evidence snippets where applicable, such as the exact suspicious form action URL or the specific Sinhala/Tamil text span that triggered the NLP alert.
- **TE5 – AI Transparency:** The system output shall clearly communicate the calculated confidence/risk score and explicitly state the model version used for the classification.
- **TE6 – Modality Contribution:** The system shall indicate the relative weight or contribution of each modality (HTML, Content, Behavior) to the final decision, allowing analysts to understand if a verdict was driven primarily by structural anomalies or semantic text.
- **TE7 – Fallback Transparency:** If a specific modality fails to extract data (e.g., JavaScript is blocked by anti-bot cloaking), the system shall explicitly state this limitation in the JSON output rather than silently defaulting to a low risk score.
- **TE8 – Analyst Override Control:** The structured output shall be designed to integrate with the platform's dashboard, providing the human analyst with the necessary evidence to confirm, reject, or override the AI-supported threat verdict.

These requirements directly correspond to the research design, which identifies multi-class classification accuracy, cross-lingual detection, and explainable, structured JSON threat intelligence as the primary contributions and evaluation measures of Component 02.

6. COMMERCIALIZATION PLAN

6.1. Target market and customer persona

The primary commercial market for Component 02 is institutional cybersecurity providers, financial institutions, e-commerce platforms, telecommunications companies, and national CERT teams that require automated, real-time web threat detection. The wider project also identifies digital payment providers and online marketplaces as potential stakeholders because of the need to protect their infrastructure from fraudulent storefronts and credential harvesting.

The primary customer persona for Component 02 is the Institutional Security Analyst or CERT Operator. Such an organization may already use traditional URL blacklists or heavy sandboxing solutions but faces difficulties detecting zero-day multi-class scams, heavily obfuscated JavaScript, and localized social-engineering attacks in low-resource languages.

A secondary user persona is the E-commerce or FinTech Platform Security Manager, who needs to continuously monitor and block fake storefronts, malicious checkout flows, and brand impersonation attempts targeting their customers.

The commercialization strategy therefore follows a Business-to-Business (B2B) and Business-to-Business-to-Consumer (B2B2C) model, where the threat intelligence API is provided to security teams and platforms, which in turn protects their everyday digital consumers from fraud.

6.2. Problem and Customer Need

Cybersecurity teams are increasingly targeted by sophisticated, multi-class web scams that evade traditional defenses. However, existing commercial tools rely primarily on reactive URL blacklists or resource-heavy OS-level sandboxes. The research problem identified for Component 02 is that blacklist-based tools cannot detect zero-day threats, while heavy sandboxes introduce unacceptable latency for real-time API scanning. Furthermore, existing content analyzers are overwhelmingly English-only, leaving a critical blind spot for localized social-engineering scams in languages like Sinhala and Tamil.

The shared commercialization analysis identifies a detection gap in web threat intelligence, particularly where institutions cannot accurately differentiate between phishing, fake e-commerce, crypto scams, and malware delivery without incurring high computational costs. It also identifies an increasing demand for explainable, multi-modal threat detection that does not rely solely on page execution.

Therefore, security teams and digital platforms have a need for a solution that can:

- detect zero-day and heavily obfuscated web threats in near real-time;
- classify threats across multiple scam categories (not just binary phishing);
- analyze client-side behavioral intent without relying on slow, heavy sandboxes;
- detect localized social-engineering lures in English, Sinhala, and Tamil;

- provide structured, explainable threat intelligence (JSON outputs) for downstream integration;
- integrate with existing SIEM, SOAR, and security dashboards via APIs.

Component 02 addresses this need by functioning as a lightweight, tri-modal semantic and behavioral analysis engine that can be integrated into existing security pipelines.

6.3. Value Proposition

The proposed product offers security institutions and digital platforms a lightweight, cross-lingual, and multi-class web threat analysis engine that can be integrated with existing security infrastructure.

The main value propositions are:

For security institutions and digital platforms

- **Zero-Day Multi-Class Detection:** Accurately classifies threats into Phishing, Fake E-commerce, Crypto/Investment Scams, and Malware-Delivery, rather than simple binary alerts.
- **Lightweight Processing:** Extracts behavioral intent statically and via lightweight DOM analysis, significantly reducing processing latency compared to heavy OS-level sandboxing.
- **Localized Scam Detection:** Identifies social-engineering lures and psychological manipulation in English, Sinhala, and Tamil, protecting local user bases.
- **Explainable Threat Intelligence:** Outputs structured, human-readable JSON findings that explain exactly *why* a page was flagged, reducing analyst investigation time.
- **Evasion Resistance:** Detects obfuscated JavaScript, hidden iframes, and form hijacking that bypass traditional static HTML analysis.
- **Seamless Integration:** Consumes raw webpage artifacts and outputs standardized JSON via API, requiring no replacement of existing security infrastructure.

For everyday digital consumers (Indirect)

- Protection against direct financial loss, identity theft, and malware infection.
- Safer digital banking and e-commerce experiences, particularly in local languages.
- Preservation of trust in digital financial services and online marketplaces.

The core commercial value is therefore not simply providing another URL scanner. It is providing a fast, explainable, and localized multi-class threat intelligence engine that closes the gap between reactive blacklists and slow sandbox execution.

6.4.Revenue Model

The proposed commercialization model will primarily follow a B2B SaaS / API licensing model.

1. API Subscription (SaaS) Security teams and digital platforms can subscribe to the threat analysis API on a monthly or annual basis. The subscription can be tiered according to:

- number of URLs/webpage artifacts scanned per month;
- required processing latency (standard vs. priority queue);
- number of supported languages for NLP analysis;
- API rate limits and concurrent request capacity.

2. Enterprise / On-Premise Licensing Larger banks, national CERTs, and telecommunications companies can license the component for dedicated, on-premise, or private-cloud deployment. Enterprise customers could receive:

- dedicated infrastructure deployment;
- customized integration with internal SIEM/SOAR platforms;
- organization-specific threat intelligence feeds;
- custom model fine-tuning for specific industry verticals;
- strict service-level agreements (SLAs) and technical support.

3. Customization and Integration Services Additional revenue can be generated through:

- system integration with existing security dashboards;
- custom language model training for specific regional dialects or industry jargon;
- development of custom JSON output schemas to match client data pipelines;
- ongoing threat intelligence consulting and model retraining.

Proposed Revenue Structure

- Core revenue: API subscription / usage-based licensing.
- Additional revenue: enterprise on-premise deployment and integration.
- Future revenue: premium localized threat intelligence feeds and custom model fine-tuning.

This model is appropriate because the proposed system is designed as a highly scalable, API-first microservice that can be consumed by various security platforms.

6.5. Cost Estimation

The main commercialization costs for Component 02 are expected to come from compute infrastructure, machine learning inference, data collection, and maintenance.

Cost Category	Expected Cost
Cloud/GPU server infrastructure (for XLM-RoBERTa inference)	Medium–High
Headless browser (Playwright) compute and scaling	Medium
Threat intelligence feed subscriptions (e.g., PhishTank, OpenPhish)	Low–Medium
Database and API gateway infrastructure	Low–Medium
Data collection and manual annotation (Sinhala/Tamil corpus)	Medium (Initial)
Software development, model retraining, and maintenance	Medium–High
Security, compliance, and isolated sandbox hosting	Medium

Table 6.1: Cost Estimation

For an initial commercial deployment, the system can reduce inference costs by using quantized or distilled versions of the XLM-RoBERTa model where acceptable, while larger enterprise deployments may require full-precision GPU inference. The exact commercial cost will depend on the volume of URLs scanned, the required latency, and the deployment architecture.

6.6. Pricing Strategy

A tiered pricing strategy is proposed to accommodate security organizations and platforms of different sizes.

Tier	Target Customer	Proposed Pricing Approach
Pilot	Small FinTech / research deployment	Low-cost or limited-period pilot (e.g., 10,000 URLs/month)
Professional	Mid-size e-commerce / digital platforms	Monthly subscription based on URL scan volume
Enterprise	Banks / large telcos / national CERTs	Customized annual license with dedicated support
Usage-Based	Platforms with highly variable demand	Pay-per-API-call pricing for burst traffic

Table 6.2: Pricing Strategy

The initial commercialization strategy should use a low-risk pilot model. A security team could first integrate the API to scan a subset of their alert queue before moving to full-scale, real-time production deployment. This reduces adoption risk because the customer can evaluate detection accuracy, processing latency, and explainability before committing to a large deployment.

6.7. Competitive Advantage

The proposed Component 02 has several potential competitive advantages over existing web threat detection solutions.

1. Tri-Modal Fusion Architecture Instead of analyzing HTML structure, text content, or JavaScript behavior in isolation, the proposed system fuses all three modalities into a unified ensemble model. This significantly reduces false positives and catches complex scams that hide malicious intent in one modality while appearing benign in another.

2. Cross-Lingual NLP Capability Support for English, Sinhala, and Tamil provides a massive competitive advantage in the South Asian market, allowing the system to detect localized social-engineering lures that bypass traditional English-only keyword filters.

3. Lightweight Behavioral Intent Extraction Unlike competitors that rely on heavy, slow OS-level sandboxes to observe page behavior, the proposed component extracts behavioral intent (e.g., form hijacking, clipboard theft) statically and via lightweight DOM event analysis. This enables near real-time processing speeds suitable for high-volume API scanning.

4. Multi-Class Scam Categorization While most commercial tools only output a binary "Phishing" or "Safe" verdict, the proposed system classifies threats into specific categories (Phishing, Fake E-commerce, Crypto Scam, Malware-Delivery), providing actionable intelligence for different response teams.

5. Explainable JSON Outputs The system does not act as a black box. It generates structured JSON findings that explicitly state the technical reasons for the verdict (e.g., "Password form submits to external domain"), allowing human analysts to verify and act on the intelligence rapidly.

6. Model-Agnostic Integration The component is designed as an analysis microservice that consumes raw webpage artifacts and outputs standardized JSON. Therefore, a security team does not need to replace their existing SIEM or dashboard infrastructure to use the proposed system.

6.8 Intellectual Property Considerations The proposed system may generate several forms of intellectual property (IP), including the software implementation, machine learning models, data pipelines, and integration architecture developed during the research.

Potential IP areas include:

- source code of the tri-modal HTML, Content, and Behavioral analysis engine;
- the fine-tuned XLM-RoBERTa model weights and methodology for Sinhala/Tamil scam-lure detection;
- the static JavaScript behavioral intent extraction and AST parsing algorithms;
- the multi-modal feature fusion and ensemble classification architecture;
- the explainable JSON threat intelligence output schema and generation mechanism;
- the localized, annotated multilingual scam-lure corpus.

The research team should maintain clear records of individual contributions and shared project components. Since Component 02 is part of a four-component research project, ownership of shared platform architecture (such as the central decision fusion engine) and jointly developed software should be determined according to SLIIT's research/project policies and any agreements established among the team members.

Third-party models, libraries, datasets, and open-source frameworks must be used according to their respective licenses. Any external transformer models (e.g., HuggingFace), headless browser tools

(e.g., Playwright), or machine learning libraries (e.g., XGBoost, Scikit-learn) incorporated into the final product should therefore be reviewed for commercial-use restrictions. The project should avoid claiming ownership of these third-party tools; instead, the potential IP contribution lies primarily in the novel tri-modal integration, the cross-lingual fine-tuning methodology, the static behavioral extraction logic, and the localized dataset developed by the team.

7. BUDGET AND JUSTIFICATION

The proposed development of Component 02 is designed to minimize additional financial costs by making use of open-source software, locally available computing resources, and free or student-accessible services where possible. The main potential expenses are associated with cloud/GPU compute for machine learning model training, data annotation for the custom multilingual corpus, and basic cloud infrastructure for API deployment.

The following budget is an initial estimate for the research and prototype-development stage. Actual costs may vary depending on the final model selection, cloud usage, and deployment requirements.

Cost Category	Estimated Cost (LKR)	Justification
Hardware	0	Existing computing equipment (personal laptop and university lab PCs) will be used for development, local model experimentation, testing, and prototype execution.
Software Licences	0	Core technologies such as Python, Playwright, BeautifulSoup, HuggingFace Transformers, XGBoost, Scikit-learn, and related development libraries are open-source.
Cloud / GPU Compute	10,000 – 20,000	Budget for GPU cloud services (e.g., Google Colab Pro, Kaggle, or basic AWS/GCP VMs) required for fine-tuning and running the XLM-RoBERTa model and scaling headless browser execution.
Data Collection & Annotation	5,000 – 10,000	Costs associated with translation API usage for data augmentation and minor stipends for peers to manually verify and annotate the custom Sinhala/Tamil scam-lure corpus.
Internet / Data Communication	5,000	Internet connectivity required for downloading large datasets, accessing cloud services, scraping URLs, and development activities.
Human Effort	Not monetized	Student development, research, model training, testing, documentation, and evaluation effort will be contributed by the researcher.
Contingency	5,000	Reserved for unexpected cloud compute overages, API rate-limit upgrades, or additional data scraping expenses.
Estimated Total	**25,000 – 40,000**	**Estimated research and prototype-development expenditure.**

Table 7.1: Budget form

7.1.Budget Justification

Hardware No dedicated hardware purchase is planned for Component 02. Development, DOM parsing, and static JavaScript analysis can be performed using existing computing resources. While XLM-RoBERTa inference and Playwright execution require decent RAM, local experimentation will utilize the researcher's personal laptop and university laboratory PCs. Cloud GPU services will be used only when local hardware is insufficient for model training.

Software Licences The proposed component primarily uses open-source technologies. The planned technology stack includes Playwright for headless browsing, HuggingFace Transformers for NLP, XGBoost and Scikit-learn for ensemble classification, and Python-based development tools. Therefore, no major commercial software licence cost is expected during the research stage.

Cloud and GPU Compute Cloud resources are required specifically for the machine learning pipeline. Fine-tuning the XLM-RoBERTa transformer model for Sinhala and Tamil, and running concurrent headless browser instances for artifact fetching, require GPU and high-memory CPU resources. A limited budget is allocated for services like Google Colab Pro or basic cloud VMs to handle these compute-heavy tasks when local hardware limits are reached. Free-tier services will be preferred where they satisfy the experimental requirements.

Data Collection and Annotation Unlike standard English-only datasets, Component 02 requires a custom multilingual scam-lure corpus. A small allocation is reserved for commercial translation API usage (to augment English phishing datasets into Sinhala and Tamil) and minor compensation for native-speaking peers to manually verify and annotate the translated text. This ensures the linguistic accuracy and contextual relevance of the NLP training data.

Internet and Communication Internet connectivity is required throughout the project for downloading large pre-trained models, accessing threat intelligence feeds (e.g., PhishTank, OpenPhish), deploying cloud instances, and conducting integration testing with the broader system.

Human Effort The researcher's development and research effort is not treated as a direct cash expense. However, the time required for literature review, data corpus annotation, system design, model training, experimentation, analysis, and documentation represents an important project resource.

Contingency A small contingency is included to accommodate unexpected expenses such as additional cloud compute usage during heavy model training phases, API rate-limit upgrades for data scraping, or unforeseen deployment requirements.

8. WORK BREAKDOWN STRUCTURE (WBS)

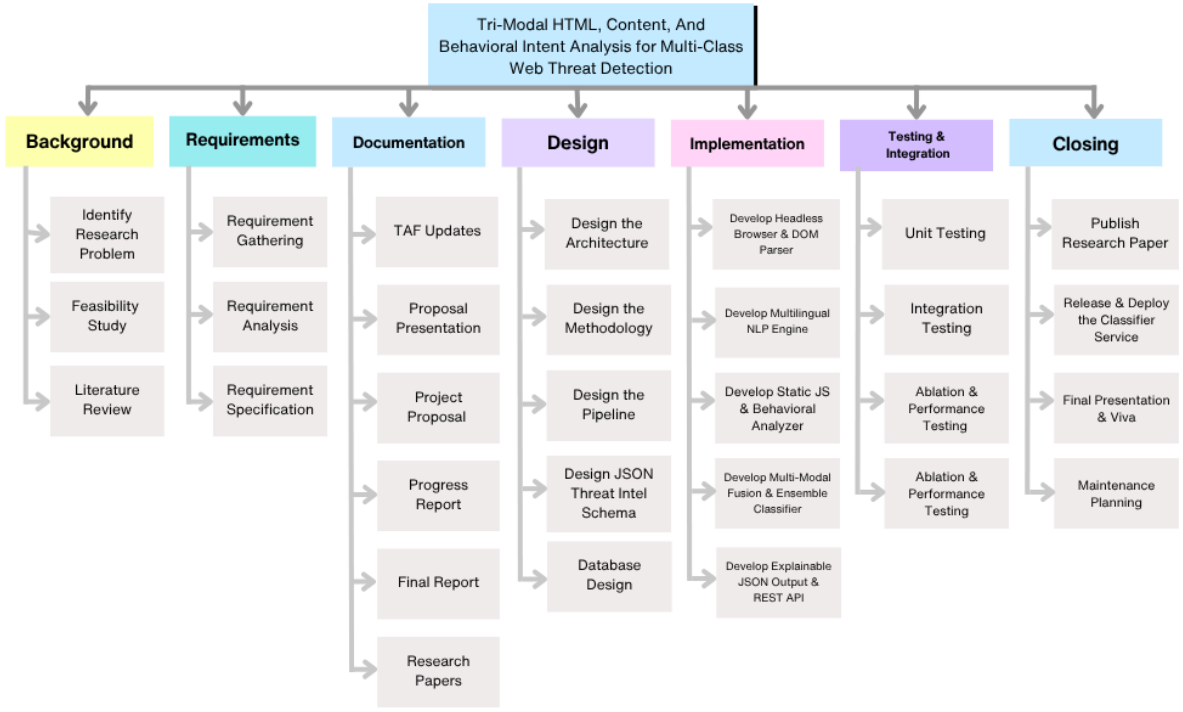


Figure 3: Work Breakdown structure

9. GANTT CHART

The Gantt chart presents the planned timeline for the tasks under Component 02: HTML, Content & Behavior Analysis, covering the period from July 2026 to February 2027. It maps the activities defined in the Work Breakdown Structure (WBS) to their expected working periods, including literature review, requirements and research design, multilingual data preparation, iterative prototype development (DOM parsing, NLP, and static JS analysis), multi-modal fusion experiments, system integration, ablation evaluation, and final dissertation preparation.



Figure 4: Gantt chart

10. REFERENCES LIST

[1] Sri Lanka Computer Emergency Readiness Team (CERT|CC), "National Cyber Security Strategy of Sri Lanka (2025–2029)," 2025. [Online]. Available: https://www.cert.gov.lk

[2] Kaspersky, "Kaspersky Security Bulletin 2024: Statistics and Threat Landscape," 2024. [Online]. Available: https://www.kaspersky.com/about/press-releases

[3] United Nations, "Sustainable Development Goals (SDGs)," 2015. [Online]. Available: https://sdgs.un.org/goals

[4] M. Khonji, Y. Iraqi, and A. Jones, "Phishing Detection: A Literature Survey," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2091–2121, 2013, doi: 10.1109/SURV.2013.030713.00220.

[5] R. S. Rao and A. R. Pais, "Detection of Phishing Websites Using an Efficient Feature-Based Machine Learning Framework," *Neural Computing and Applications*, vol. 31, pp. 3851–3873, 2019, doi: 10.1007/s00521-017-3305-0.

[6] D. Zou, G. Wang, J. Xu, S. Liu, Y. Zhang, Y. Hao, X. Han, and H. Jin, "A Fine-Grained Deep Learning Approach for Phishing Web Page Detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 2, pp. 1120-1133, 2020, doi: 10.1109/TDSC.2020.3010222.

[7] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, É. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised Cross-lingual Representation Learning at Scale," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8440–8451, doi: 10.18653/v1/2020.acl-main.747.

[8] S. Ranathunga, E. S. A. Lee, M. Skenduli, R. Sheehan, M. Alam, and R. Kaur, "Neural Machine Translation for Low-Resource Languages: A Survey," *ACM Transactions on Asian and Low-Resource Language Information Processing*, vol. 22, no. 4, pp. 1–43, 2023, doi: 10.1145/3584376.

[9] A. Atondo Siu, "BERT-based Detection of Cryptocurrency Scams," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2024, pp. 112-119.

[10] F. M. F. de Jong, "Detecting Romance Scams: Image Reuse and Conversational Language Analysis," Master's thesis, Delft Univ. of Technology, Delft, Netherlands, 2019.

[11] S. Lekies, B. Stock, and M. Johns, "25 Million Flows Later: Large-scale Detection of DOM-based XSS," in *Proc. 2013 ACM SIGSAC Conf. Computer & Communications Security (CCS)*, 2013, pp. 1123–1134, doi: 10.1145/2508859.2516681.

[12] E. Ugarte-Pedrero, D. Balzarotti, F. Maggi, and S. Zanero, "SoK: A Comparative Study of JavaScript Malware Detection Techniques," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2016, pp. 1-10, doi: 10.1109/SPW.2016.15.

[13] A. Kharraz, W. Robertson, D. Balzarotti, L. Bilge, and E. Kirda, "Out of Control: A Security Analysis of Web-based Payment Systems," in *Proc. USENIX Security Symposium*, 2015, pp. 103-118.

[14] G. Rydstedt, E. Bursztein, D. Boneh, and C. Jackson, "Busting Frame Busting: A Study of Clickjacking Vulnerabilities at Popular Sites," in *Proc. IEEE Security and Privacy Workshops*, 2010, pp. 1-10, doi: 10.1109/SPW.2010.15.

[15] D. Sánchez-Paniagua, R. M. Gasca, and J. A. Á. Olmo, "Detecting Fake E-commerce Websites using HTML and Text Features," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2021, pp. 450-458.

[16] A. Oest, Y. Safaei, A. Doupé, and G.-J. Ahn, "Inside a Phish's Mind: An Investigation into Phishing Kit Ecosystems," in *Proc. ACM APWG eCrime Researchers Summit*, 2017, pp. 1-12.

[17] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[18] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[19] Microsoft, "Playwright: Cross-Browser Automation Framework," 2024. [Online]. Available: https://playwright.dev

[20] Google, "Google Safe Browsing: Protecting Users from Malicious Websites," 2024. [Online]. Available: https://safebrowsing.google.com

[21] OpenPhish, "Phishing Intelligence Feed and Threat Data," 2024. [Online]. Available: https://openphish.com

[22] Microsoft, "Microsoft Defender SmartScreen," 2024. [Online]. Available: https://learn.microsoft.com/en-us/windows/security/operating-system-security/virus-and-threat-protection/microsoft-defender-smartscreen

11. APPENDICES

Appendix 1 - Supporting Proposal Evidence

Appendices 1.Supporting Proposal Evidence

A. Literature Comparison

The following matrix summarizes the main literature used to establish the Component 02 research gap. The comparison focuses on whether each study addresses DOM/HTML structural analysis, multilingual NLP (specifically low-resource languages), static JavaScript behavioral intent extraction, and multi-class scam categorization.

Study	DOM/HTML Structural Analysis	Multilingual NLP (Sinhala/Tamil)	Static JS/Behavioral Intent Analysis	Multi-Class Scam Categorization	Key Limitation Relevant to C2
Khonji et al. [1]	✓	✗	✗	✗	Foundational survey; lacks unified multimodal fusion or behavioral analysis.
Rao & Pais [2]	✓	✗	✗	✗	Uses shallow HTML features for binary classification; ignores JS behavior and multilingual content.
Conneau et al. [4]	✗	✓	✗	✗	Introduces XLM-RoBERTa, but not fine-tuned for cybersecurity or scam-lure detection.
Lekies et al. [7]	✓	✗	✓	✗	Large-scale static JS dataflow tracking, but focused on XSS, not scam-specific behaviors (e.g., form hijacking).
Ugarte-Pedrero et al. [8]	✗	✗	✓	✗	AST-based malicious JS detection, but treats all threats as binary, lacking multi-class scam typology mapping.
Sánchez-Paniagua et al. [10]	✓	✗	✗	✓	Fake e-commerce detection using HTML/text, but misses post-load scripted behaviors and lacks multilingual support.
Oest et al. [11]	✓	✗	✗	✓	Analyzes phishing kit structures, but is retrospective/manual and lacks real-time

					automated NLP integration.

Table 11.1: Literature Comparison for Component 02

The comparison supports the finalized research position that the gap lies in the limited integration of these capabilities. While individual modalities (DOM, NLP, or JS analysis) have been studied, no existing lightweight pipeline fuses them to detect multi-class, localized (Sinhala/Tamil) web scams without relying on heavy OS-level sandboxing.

B. Preliminary Data Requirements

A single, pre-labeled public dataset covering all required modalities and languages does not exist. The finalized C2 design instead defines the minimum information that the synthesized experimental datasets must provide.

Data Element	Requirement	Purpose
HTML Source & DOM Structure	Required	Extraction of structural features (forms, hidden elements, iframes, deceptive UI).
JavaScript Code (Inline/External)	Required	Static AST parsing, obfuscation scoring, and dataflow tracking for behavioral intent.
Visible Webpage Text	Required	NLP-based detection of social-engineering lures, urgency triggers, and fake warnings.
Form Actions & Endpoints	Required	Detection of form hijacking and credential harvesting to external/suspicious domains.
Language (EN, SI, TA)	Required	Training and evaluating the cross-lingual NLP module for localized scam detection.
Threat Category Label	Required	Multi-class classification (Phishing, Fake E-commerce, Crypto Scam, Malware-Delivery, Legitimate).
Standardized Feature Vectors	Derived	Fusion of DOM vectors, XLM-RoBERTa embeddings, and binary behavioral flags for ML training.

Table 11.2: Preliminary Data Requirements for Component 02

The selected sources (PhishTank, OpenPhish, Tranco Top 1M, ELFW-2031, and custom scraped local data) will provide sufficient diversity for chronological aggregation, training, validation, and ablation testing.

Appendix 2 - Mandatory Proposal Evidence and Decision Log

Appendices 2. Mandatory Proposal Evidence and Decision Log

The proposal evidence and decision log records the main reasoning used to refine Component 02 from the initial topic-assessment scope into the finalized proposal design. The current proposal template requires the decision log to connect the identified claim or problem with evidence, considered alternatives, and the resulting decision.

Date	Claim / Problem	Evidence Collected	Alternatives Considered	Decision Made	Supervisor Discussion
08/07/2026	Component 02 required further refinement to avoid overlap with the heavy OS-level sandbox execution (Component 04) and clearly define its unique contribution.	Topic Assessment Form (TAF) feedback and initial project architecture review highlighting the 4-component structure.	Retain broad scope including dynamic sandbox execution; OR refine to focus strictly on lightweight static/lightweight behavioral analysis.	Refine C2 to focus exclusively on lightweight static HTML, Content, and Behavioral Intent analysis (DOM/JS/NLP), leaving heavy OS-level sandboxing to Component 04.	Panel feedback recorded and incorporated. Scope boundaries clarified.
20/08/2026	The research gap could not defensibly claim that NLP for phishing detection had not been studied previously.	Verified literature showed extensive work on English-only NLP phishing detection.	Claim a completely new research area; OR define a narrower integration gap focused on multilingual and behavioral fusion.	Use a limited-integration gap focused on fusing cross-lingual NLP (Sinhala/Tamil) with static JS behavioral intent for multi-class scam detection.	Gap positioning finalized; confirmation pending.
25/08/2026	Public datasets for Sinhala and Tamil phishing/scam text are highly limited or non-existent.	Literature search and preliminary dataset exploration confirmed severe scarcity of low-resource cybersecurity corpora.	Rely only on English datasets; OR build a custom corpus via translation and local scraping.	Construct a custom multilingual scam-lure corpus using LLM-assisted translation of existing datasets, augmented with manually verified local social media scam reports.	Data collection strategy finalized; ethical anonymization confirmed.
30/09/2026	Raw machine learning classification scores alone would not provide the structured, actionable intelligence required by security analysts.	User requirement analysis and stakeholder persona definitions (Institutional Security Analyst).	Output only a binary/multi-class label and probability score; OR generate structured, explainable findings.	Generate a structured JSON output containing the verdict, risk score, and human-readable "findings" (e.g., "Password form submits to external domain").	Output schema finalized; alignment with Decision Fusion Engine confirmed.
05/09/2026	Heavy OS-level sandboxing is too slow for real-time API scanning, but static analysis misses dynamic behaviors.	Performance benchmarks of existing sandbox solutions vs. static AST/dataflow analysis papers (e.g., Lekies et al., Ugarte-Pedrero et al.).	Use a full OS-level sandbox for all behavioral analysis; OR rely solely on static HTML features.	Implement "lightweight static behavioral intent extraction" (AST parsing, dataflow tracking, DOM event analysis) to detect form hijacking and clipboard theft without heavy sandbox execution.	Technical approach validated; ensures clear separation from Component 04 (Sandbox).

Table 11.3: Mandatory Proposal Evidence and Decision Log

Appendix 3 - AI Use Disclosure

Appendices 3. AI Use Disclosure

AI Tool	Purpose of Use	What Was Generated / Assisted	How Output Was Verified	Final Student Contribution
ChatGPT / Claude	Used to support understanding of research concepts, compare methodology options, organize the proposal according to the current SLIIT template, improve academic wording and clarity, assist with literature organization, and support planning of tables, architecture descriptions, evaluation metrics, budget, and appendices.	Draft wording, structural suggestions, literature comparison matrices, research-planning suggestions, methodology and evaluation explanations, reference organization, and prompts for further verification.	Outputs were rigorously checked against the current 2026 Project Proposal Report template, Final Proposal Presentation, TAF document, original peer-reviewed research papers, DOI/publisher records, and previously confirmed project decisions. Draft text was reviewed, fact-checked, and revised before being accepted.	The student defined the research problem and scope, selected the final component boundaries and methodology, reviewed all evidence, verified every reference, accepted or rejected AI suggestions, revised the report content, and remains fully responsible for the accuracy, originality, and defense of the final submitted work.

Table 11.4: AI Use Disclosure